EVALUACIÓN PRÁCTICA INTEGRAL: BD & API REST

Examen Práctico: Sistema Logístico LogiExpress

Administración de Bases de Datos
Motor: MySQL 8.x
ORM: TypeORM (TypeScript)

> 📌 **INSTRUCCIONES GENERALES Y REGLAS DE ENTREGA**
>
> 1. Lee con atención el contexto del negocio antes de comenzar la codificación.
> 2. PROCEDIMIENTOS ALMACENADOS (Procedures): Se crean en MySQL y se prueban directamente desde DBeaver ejecutándolos con parámetros de consulta. NO se llaman desde la API REST.
> 3. TRIGGERS: Se definen en MySQL para ejecutarse automáticamente en segundo plano ante las operaciones de actualización o inserción realizadas por la API.
> 4. API TYPEORM (TypeScript): Implementa los 6 endpoints requeridos. Al realizar inserciones o modificaciones, la API detonará los triggers y deberá capturar los errores lanzados (ej. HTTP 400).
> 5. ENTREGABLES: Archivo `.sql` con el DDL, procedimientos y triggers + Proyecto en TypeScript (.zip) estructurado con entidades, controladores y servicios.

1. Contexto de la Problemática: Empresa LogiExpress

LogiExpress es una empresa de paquetería y logística regional en rápida expansión. Actualmente enfrentan fallas operativas graves debido a la falta de automatización y validaciones en su base de datos. La empresa te ha contratado para rediseñar la arquitectura técnica combinando MySQL 8.x y una API REST en TypeScript con TypeORM.

Resumen de las 4 Fallas Operativas Críticas a Resolver:

Falla Operativa	Impacto en el Negocio	Solución Técnica Requerida
1. Descontrol de Capacidad	Almacenes reciben paquetes sin validación, generando sobrecupo, extravíos y retrasos.	Trigger BEFORE INSERT (`before_insert_paquete_almacen`) que valida espacio disponible o lanza `SIGNAL SQLSTATE '45000'`.
2. Ajustes de Costo sin Auditoría	Cambios manuales en precios sin registro de responsable ni monto original, provocando descuadres.	Trigger AFTER UPDATE (`after_update_envio_costo`) que guarda en `auditoria_envios` el valor `OLD`, `NEW`, usuario y fecha.
3. Inconsistencia de Estado	Repartidores registran entregas en rastreo pero el estado global del envío no se actualiza.	Trigger AFTER INSERT (`after_insert_historial_rastreo`) que al recibir 'ENTREGADO' actualiza la tabla de envíos.
4. Sobrecarga por Consultas	Consultas pesadas de reportes desde el sistema web saturan los servidores operativos.	Stored Procedures en MySQL (`sp_reporte_mensual_cliente` y `sp_obtener_hoja_ruta_envio`) ejecutados solo en DBeaver.

2. Tablas del Sistema a Modelar (7 Tablas)

El esquema relacional de la base de datos MySQL debe estar compuesto estrictamente por las siguientes 7 tablas en 3FN:

- **clientes:** Guarda información del cliente, estado (ACTIVO/INACTIVO) y límite de crédito.
- **direcciones:** Almacena direcciones de origen y destino asociadas a cada cliente.
- **envios:** Registra el envío general, remitente, destinatario, costo_total, fecha y estado global (REGISTRADO, EN_TRANSITO, ENTREGADO, CANCELADO).
- **paquetes:** Detalla cada paquete contenido en un envío (peso, dimensiones, contenido).

- **almacenes:** Centros de distribución con su nombre, ubicación y capacidad_maxima de paquetes.
- **historial_rastreo:** Bitácora de seguimiento con fecha, ubicación_actual, estado_evento y observaciones.
- **auditoria_envios:** Tabla de control donde se registran los cambios de costos (costo_anterior, costo_nuevo, usuario, fecha_modificacion).

3. Requerimientos Prácticos del Examen (10 Puntos / 100 PTS)

#	Descripción del Requerimiento y Formato de Entrega	Valor
1	**Diseño de Base de Datos y Normalización (Script SQL)** Diseñe el script DDL en MySQL para las 7 tablas mencionadas. Aplique la 3ª Forma Normal (3FN), definiendo explícitamente llaves primarias (PK), foráneas (FK) y tipos de datos adecuados (DATETIME, DECIMAL, ENUM).	**10 PTS**
2	**Modelado de Entidades y Relaciones (TypeScript / TypeORM)** Cree los archivos de TypeScript con decoradores `@Entity()` (`Cliente`, `Direccion`, `Envio`, `Paquete`, `Almacen`, `HistorialRastreo`, `AuditoriaEnvio`) configurando campos y relaciones (`@ManyToOne`, `@OneToMany`).	**10 PTS**
3	**Procedure SQL 1: Reporte Mensual de Cliente (DBeaver / MySQL)** Cree el Stored Procedure `sp_reporte_mensual_cliente(p_cliente_id, p_mes, p_anio)`. Se ejecuta en DBeaver pasándole parámetros para consultar facturación total, envíos entregados y tiempo promedio de entrega.	**10 PTS**
4	**Procedure SQL 2: Hoja de Ruta e Historial (DBeaver / MySQL)** Diseñe el Stored Procedure `sp_obtener_hoja_ruta_envio(p_envio_id)`. Se ejecuta en DBeaver recibiendo el ID del envío y retorna el desglose del paquete, remitente/destinatario y el historial cronológico de rastreo.	**10 PTS**
5	**Triggers de Base de Datos (MySQL)** Cree en MySQL los 3 triggers que reaccionan a las operaciones de la API: • `after_update_envio_costo`: Guarda cambios de `costo_total` en `auditoria_envios`. • `before_insert_paquete_almacen`: Cancela con `SIGNAL SQLSTATE '45000'` si el almacén superó la capacidad. • `after_insert_historial_rastreo`: Al recibir estado 'ENTREGADO', actualiza el estado global en `envios`.	**10 PTS**
6	**Endpoint 1: POST /api/envios (API TypeScript / TypeORM)** Implemente en TypeScript el endpoint `POST /api/envios` para dar de alta un envío con sus paquetes iniciales utilizando repositorios o QueryRunner de TypeORM.	**10 PTS**
7	**Endpoint 2: PUT /api/envios/:id/costo (API TypeScript / Detona Trigger Auditoría)** Desarrolle el endpoint `PUT /api/envios/:id/costo`. Al actualizar el costo mediante TypeORM, debe dispararse el Trigger `after_update_envio_costo` registrando la auditoría.	**10 PTS**
8	**Endpoint 3: POST /api/almacenes/:id/paquetes (API TypeScript / Detona Trigger Capacidad)** Implemente `POST /api/almacenes/:id/paquetes`. Al insertar desde TypeORM se ejecuta el Trigger `before_insert_paquete_almacen`. El API debe capturar el error `45000` si sobrepasa la capacidad y responder HTTP 400 Bad Request.	**10 PTS**
9	**Endpoint 4: POST /api/envios/:id/rastreo (API TypeScript / Detona Trigger Estado)** Cree el endpoint `POST /api/envios/:id/rastreo`. Al insertar la ubicación en TypeORM, el Trigger `after_insert_historial_rastreo` actualizará automáticamente el estado del envío a 'ENTREGADO'.	**10 PTS**
10	**Endpoints 5 y 6: Consultas Relacionales (API TypeScript / TypeORM)** Implemente `GET /api/envios` (envíos activos con clientes/paquetes) y `GET /api/clientes/:id/envios` (historial por cliente) utilizando `Repository.find()` o `createQueryBuilder()`.	**10 PTS**

> 📋 **RESUMEN DE LOS 6 ENDPOINTS REQUERIDOS**
>
> 1. POST /api/envios → Registra un nuevo envío y guarda sus paquetes iniciales en TypeORM.
> 2. PUT /api/envios/:id/costo → Modifica el costo total → Detona Trigger `after_update_envio_costo` (Auditoría).

3. POST /api/almacenes/:id/paquetes → Asigna paquete → Detona Trigger `before_insert_paquete_almacen` (Capturar error 45000 -> HTTP 400).
4. POST /api/envios/:id/rastreo → Agrega rastreo → Detona Trigger `after_insert_historial_rastreo` (Actualiza estado a ENTREGADO).
5. GET /api/envios → Consulta de envíos activos con TypeORM QueryBuilder.
6. GET /api/clientes/:id/envios → Historial de envíos vinculados a un cliente específico.

4. Rúbrica de Criterios de Evaluación

Módulo Evaluado	Criterios Técnicos de Aceptación	Porcentaje
Diseño & Procedures SQL (DBeaver)	Normalización en 3FN de las 7 tablas. Correcta sintaxis de los 2 Procedures probados mediante `CALL` con parámetros en DBeaver.	**30%**
Triggers de BD (MySQL)	Sintaxis precisa en MySQL de los 3 Triggers (`SIGNAL SQLSTATE '45000'`, auditoría en `auditoria_envios` y actualización automática de estado).	**30%**
API TypeScript & 6 Endpoints	Definición de Entidades decoradas en TypeORM, construcción de los 6 Endpoints HTTP y captura correcta de la excepción HTTP 400 ante triggers.	**40%**